ALGORITHM WHITEPAPER

MiniDB Studio

A bilingual storage-engine and desktop database studio whitepaper for a pure C core, rule-based query planning, B+ Tree traversal, WAL recovery, and state-driven desktop observability.

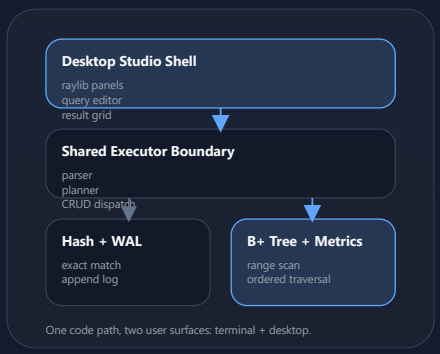
This package documents the current architecture of MiniDB Studio as it exists in the codebase today: reusable engine modules, index-aware query execution, deterministic assistant suggestions, and raylib-based desktop synchronization on top of the same executor boundary.

- Pure C11 Core
- Hash Index
- B+ Tree
- CSV Snapshot
- WAL Replay
- raylib Desktop

ENGINE BOUND	INDEX PATHS	RECOVE MODEL	DOCUM SET
Reus	3	CSV + WAL	EN + KO

Architecture Snapshot

A shared execution core keeps query semantics, recovery rules, and telemetry consistent across terminal and desktop surfaces.



AUDIENCE

Engineering portfolio reviewers, interviewers, and systems-programming peers.

SCOPE

Parser, planner, execution engine, recovery, desktop synchronization, and performance telemetry.

POSITIONING

A lightweight database management studio with explainable internals and demo-ready visuals.

SECTION 02

Project Overview

MiniDB Studio combines an in-memory record engine, rule-based optimizer, and desktop control surface without splitting the storage core from the UI shell. The result is a portfolio project that behaves like a miniature database studio rather than a parser demo.

System Scope

- A `C11` storage core owns records, indexes, recovery state, and execution statistics.
- Both the terminal console and the raylib desktop app reuse the same `parse -> plan -> execute` boundary.
- Exact lookups are accelerated through hash indexes, while ordered access and range scans flow through B+ Trees.
- Persistence is modeled as snapshot + log replay, which makes recovery explainable in a short interview walkthrough.

Module Inventory

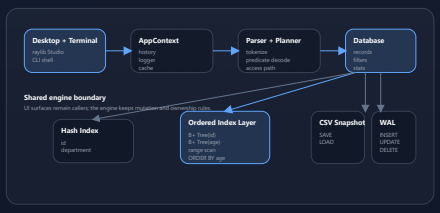
MODULE	PRIMARY RESPONSIBILITY	RE
`parser.c`	Tokenization, validation, and command decoding.	par
`planner.c`	Rule-based access-path selection and sort pushdown checks.	pla
`database.c`	CRUD dispatch, filter evaluation, index maintenance, and query statistics.	dat dat
`bptree.c`	Ordered access and range traversal.	bpt bpt
`wal.c`	Append-only mutation logging and replay.	wal
`studio_app.c`	Desktop state transitions and metric streaming.	stu stu

SECTION 03

Core Storage Engine Architecture

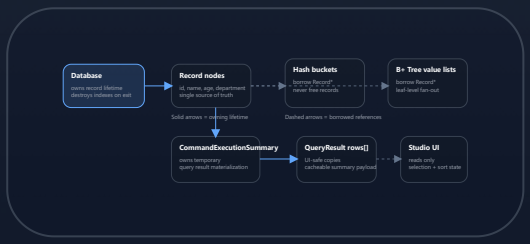
The most important design choice is ownership isolation. Record lifetime stays inside `Database`, secondary indexes only borrow `Record*` references, and UI-facing summaries materialize temporary result rows so the desktop layer never becomes the owner of storage memory.

Execution Layers



The same execution path serves the terminal and the desktop studio. That separation preserves the pure C engine while allowing new presentation layers.

Memory Ownership Model



This ownership split is what makes LOAD safe: a temporary database can be built, validated, and only then swapped into `AppContext`.

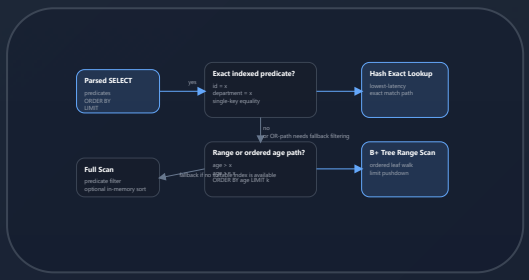
> Design invariant: mutation paths update indexes and persistence metadata inside the engine before UI code reads the new state. The raylib layer never frees storage objects directly.

SECTION 04

Query Planner & Optimizer Algorithm

The optimizer is intentionally lightweight and explainable. It does not rely on a cost model; instead, it applies deterministic rules that map query shape to the best available access path, then annotates the UI with the chosen plan string.

Planner Decision Flow



The planner emits plan labels such as `B+ Tree Range Scan` or `Hash Exact Lookup`, which are surfaced directly in the desktop inspector.

Rule Set and Planner Effects

QUERY SHAPE	PRIMARY PLAN	PLANNER SIDE EFFECT
`SELECT WHERE id = 42`	Hash exact lookup or B+ exact lookup	Scans a single bucket or key slot and reports a low row scan count.
`SELECT WHERE department = Oncology`	Hash exact lookup	Uses the department hash index and filters only the candidate chain.
`SELECT WHERE age > 30`	B+ Tree range scan	Starts at the first qualifying leaf and walks the linked leaf chain.
`SELECT ORDER BY age LIMIT 10`	B+ Tree ordered scan	Suppresses extra sort work when the ordered

QUERY SHAPE	PRIMARY PLAN	PLANNER SIDE EFFECT
		leaf stream already matches the requested ordering.
Mixed AND predicate	Indexed seed + post-filter	Uses the best seed path, then evaluates remaining predicates in memory.
Unsupported or broad OR	Full scan	Falls back to a complete record walk so correctness is preserved.

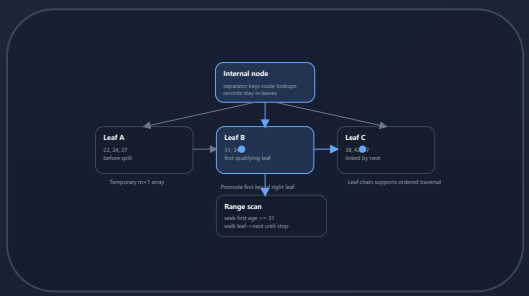
The planner also decides whether a later in-memory sort is still required. If leaf traversal already provides the requested `ORDER BY age`, the engine avoids extra reorder work.

SECTION 05

B+ Tree Traversal Logic

The ordered index layer exists to make age-based range scans and ordered traversal feel like storage-engine work rather than linear filtering. The tree keeps all records in leaves, stores separator keys in internal nodes, and links leaves through a forward chain for fast sequential access.

Split and Range Path



The current implementation comments the split logic directly in `bptree.c` so leaf and internal promotion rules are visible to reviewers.

Operational Semantics

- `bptree_insert` descends to the correct leaf, inserts in sorted order, and bubbles a split upward only when a node overflows.
- Leaf split logic uses a temporary array with one extra slot, keeps the lower half in place, moves the upper half into a new right sibling, and promotes the first key of that right sibling.
- Internal split logic promotes the middle separator upward while redistributing children so parent routing stays correct after the split.
- Non-unique age keys are stored as value lists attached to a single leaf key, which keeps duplicate ages ordered without duplicating separators.
- Deletion uses borrow-or-merge rebalancing helpers so underflowing nodes repair themselves before the tree returns control to the caller.

OPERATION	TYPICAL COMPLEXITY	NOTES
Exact lookup	`O(log_B n)`	Descend through

OPERATION	TYPICAL COMPLEXITY	NOTES
		separators one leaf.
Range scan	O(log_B n + k)	Seek once then strea through le links.
Insert	O(log_B n)	Split only overflowin nodes.
Delete	O(log_B n)	Borrow/m on underf

SECTION 06

WAL and Snapshot Recovery Flow

Persistence is modeled as a checkpointable snapshot plus an append-only mutation log. This keeps on-disk behavior easy to reason about: SAVE emits a stable CSV image, and startup recovery replays any later mutations from the WAL.

Recovery Sequence

The executor loads into a temporary `Database`, replays `data/db.log`, and only then swaps the restored engine into the active context.

Recovery Guarantees

PHASE	CURRENT ALGORITHM	WHY IT MATTERS
Mutation logging	Append `INSERT|...`, `UPDATE|...`, or `DELETE|...` records.	Every change after the last snapshot has replayable representatic
Snapshot save	Write all rows to CSV, then checkpoint the WAL.	The CSV becomes a concise resto point and the log stays bounded.
Restore	Load CSV into a temporary engine, replay WAL, rebuild indexes as records are inserted.	Indexes and row store are restored through the same code paths used ir normal execution.
Activation	Assign the restored database into `AppContext` only after success.	Prevents partial recovery fron corrupting th active sessior

Because replay calls the normal database mutation APIs, index consistency is not a separate recovery step; it is a side effect of reusing the same insert/update/delete logic.

SECTION 07

AI Query Assistant Parsing Flow

The current desktop assistant is intentionally deterministic. It is not an LLM runtime; instead, it synthesizes valid SQL-like suggestions from row context, prior execution plans, and canned demo heuristics, then routes those suggestions back through the same parser used for typed text.

Current Assistant Pipeline

Examples include `SELECT WHERE id = ...` and `SELECT WHERE age > ... ORDER BY age LIMIT` when a row or plan indicates that the B+ Tree would be a good fit.

Assistant Design Contract

- Selected-row context seeds deterministic query templates that are guaranteed to be syntactically valid for the current grammar.
- Recent execution plans bias suggestions toward indexed alternatives when a previous command triggered a full scan.
- The editor remains the visible source of truth, so users can inspect or modify a generated command before execution.
- Because assistant output flows back through `parse_command`, validation, logging, benchmarking, and caching all continue to work unchanged.

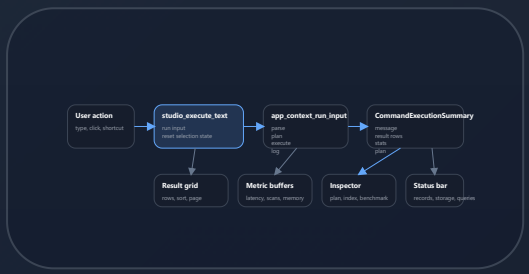
Future natural-language parsing belongs in the roadmap, not in the current-state architecture claim. The existing implementation already creates a useful “AI assistant” surface by turning engine state into context-aware query suggestions.

SECTION 08

Desktop UI-State Synchronization

The desktop application stays responsive because UI state is thin and derived. Query execution mutates engine state inside `AppContext`, then the desktop layer consumes summarized outputs for the editor, grid, inspector, metrics dashboard, and footer.

State Transition Graph



The desktop shell does not rebuild storage views from scratch. Instead, it reacts to a compact execution summary and a snapshot of dashboard status from the engine.

Synchronization Points

UI EVENT	PRIMARY FUNCTION
Execute query	`studio_execute_text`
Engine dispatch	`app_context_run_input`
Metrics update	`studio_push_metrics`

UI EVENT	PRIMARY FUNCTION
Grid sort	`result_grid_sort`
Footer refresh	`app_context_snapshot_status`

This flow is why the studio can stay visually rich without turning into a second storage layer. The UI owns presentation state; the engine owns truth.

SECTION 09

Performance Dashboard Metrics

MiniDB Studio exposes internal engine behavior as presentation-friendly telemetry. Each executed command updates a compact set of metric series so the desktop dashboard can explain not just results, but also the cost profile of how those results were produced.

Metric Data Flow

The dashboard visualizes engine-side signals that already exist in execution summaries, so the desktop layer is reading metrics rather than inventing them.

Metric Definitions

METRIC	SOURCE FIELD
Query latency trend	`last_elapsed_ms`
Rows scanned	`summary.stats.rows_scanned`
Memory usage	`database_estimate_memory_u`
Cache hit ratio	`query_cache_hits / query_cache_lookups`
Index usage frequency	Plan counters keyed by chosen p

METRIC	SOURCE FIELD

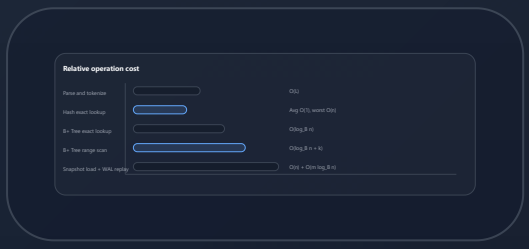
The dashboard is intentionally lightweight: tiny chart buffers, no heavy telemetry subsystem, and all values derived from already-existing execution accounting.

SECTION 10

Complexity Analysis

The engine is small enough to explain asymptotically, but rich enough to show real trade-offs between exact lookup structures, ordered traversals, and recovery work. The chart below uses relative bar lengths to show the operational cost profile reviewers should expect.

Relative Cost Profile



Range scans become attractive when result ordering matters, even if the point lookup case is faster in the hash index.

Complexity Table

OPERATION	COMPLEXITY	WHY IT MATTERS
Tokenize + parse	O(L)	Linear in c text lengt the front e predictabl
Hash exact lookup	O(1) average	Best path direct id o departme equality fil
B+ Tree exact lookup	O(log_B n)	Stable ord search wit small tree
B+ Tree range scan	O(log_B n + k)	One seek followed b chain strea
Insert / delete with index maintenance	O(log_B n)	Mutation dominate Tree maint after reco lookup/all
Snapshot load	O(n)	All rows a reconstru through n insert pat

OPERATION	COMPLEXITY	WHY IT MATTERS
WAL replay	`O(m log_B n)`	Recovery scales with length and rebuild wo
Result-grid sort fallback	`O(r log r)`	Used only the planne cannot pre the desire natively.

SECTION 11

Future Scaling Roadmap

The current architecture is deliberately compact, but its seams already point toward larger storage-engine work. The roadmap below keeps the pure C core intact while extending it toward disk management, richer planning, and more capable assistant behavior.

NEAR-TERM

Local engine depth

Introduce slotted-page serialization, richer index statistics, and more selective predicate normalization while preserving the current in-memory API surface.

MID-TERM

Storage realism

Add a buffer-pool abstraction, asynchronous checkpointing, composite indexes, and more explicit recovery metadata so SAVE/LOAD can grow into page-oriented persistence.

LONG-TERM

DB studio evolution

Layer MVCC, a cost-based optimizer, and validated natural-language-to-query conversion on top of the existing parser boundary without rewriting the storage engine.

Roadmap Principle

Every next step should keep the current strength of the project intact: explainable internals, reusable boundaries, and a UI that visualizes real engine behavior instead of hiding it.

SECTION 12

Final Engineering Summary

MiniDB Studio succeeds as a portfolio system because the engine, optimizer, recovery path, and desktop shell tell one coherent story. It is small enough to present in a few minutes and deep enough to discuss indexing, ownership, and execution trade-offs with confidence.

Why the design holds together

REUSABLE CORE

The same parser, planner, and database code power both the console interface and the desktop studio, which makes the architecture feel intentional rather than forked.

EXPLAINABLE PERFORMANCE

Hash lookups, B+ Tree traversals, scan counts, and cache hits all surface clearly enough to discuss the query path chosen for a single command.

RECOVERY CLARITY

CSV snapshots plus WAL replay create a clean mental model for

Portfolio framing

MiniDB Studio demonstrates the kind of engineering judgment interviewers look for in systems work: explicit ownership rules, data-structure-aware query planning, predictable recovery, and a presentation layer that makes low-level decisions easy to inspect.

Three-minute pitch: pure C storage core, hash + B+ Tree indexing, WAL-backed recovery, shared executor boundary, and a desktop studio that visualizes query plans and runtime metrics.

durability, restore safety, and index reconstruction through normal mutation paths.